AI Advances Briefs

Member-only story

Embedded Ai | Claude | Mods | Ai Development | Claude Code

Building an Embedded Firmware Dashboard Mod for Claude Code

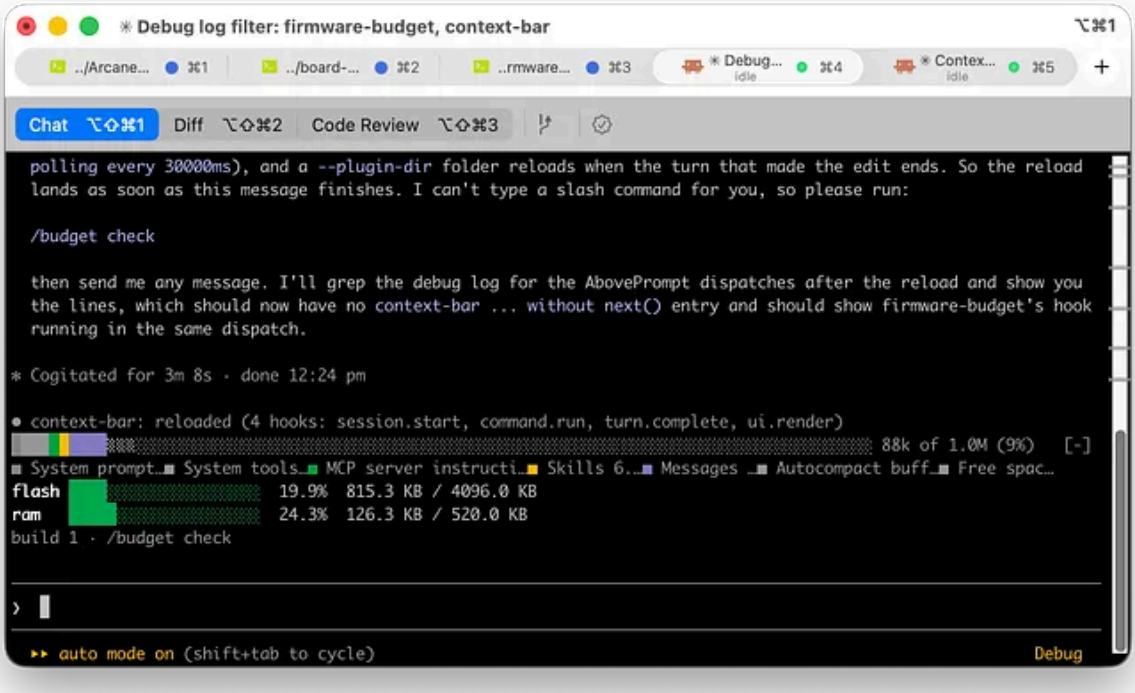
After writing a mod in Claude to track the state of the context window, I started thinking about where else this might be useful.



David Such Follow 7 min read · 1 hour ago

100

A microcontroller does not have a swap file. When your compiled firmware is bigger than flash memory, there is no work around, you need to cut out some code or choose a different MCU. If it outgrows RAM, the linker may tell you, or more likely the stack grows into the heap and the board crashes with nothing in the backtrace to tell you why.



iTerm screenshot by the author

In a recent project I did on a Pico 2 there was 4 MB of flash and 520 KB of SRAM to share between a neural network, its tensor arena, the audio buffers, the display driver and the SDK itself. Most of the projects in my book on Embedded AI spend a page or two working out which processor to use for each project. Getting a model onto the chip is the easy bit. Getting it to fit beside everything else — not so much. So the two memory numbers I'm interested in seeing on every build are flash and RAM used.

```
flash ████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  19.9%  815.3 KB / 4096.0 KB
ram   ████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  24.3%  126.3 KB / 520.0 KB
build 1 · /budget check
```

One thing these bars won't show is what the firmware does once it runs. `size` reports what the linker included: code, constants, initialised and zero-initialised data, and the SDK's minimum reservations for heap and stack. It

knows nothing about `malloc` calls at runtime, a tensor arena that is sized in code, or how deep the stack goes when an interrupt is generated.

The Dashboard Mod

To generate the dashboard bars I created a Claude Code mod called *firmware-budget*. After every build it runs the toolchain's `size` on the linked ELF and draws two bars above the prompt: flash and RAM, as a fraction of the budget, with the change since the last build. If the last build is over budget, it refuses `git commit`. The refusal happens inside Claude Code, before the permission check, and you can't argue your way past it or modify *CLAUDE.md* to get it to commit. The only ways through are to shrink the firmware or to raise the memory budget in `/config`.

Embedded AI: Bonus Projects and Free Technical Articles

Embedded AI: Intelligence at the Deep Edge is now available from No Starch Press. Subscribe to the Embedded AI...

embedded-ai.kit.com

The images in this article are from the Chapter 10 project from *Embedded AI*: Noise Suppression using RNNoise running on a Pico 2 using a PDM microphone, and displaying the spectrogram on an OLED. The measured numbers come from the linker, which I use to check against what I expect to happen.

Why a Mod and Not a Hook

A mod is a plugin whose code runs inside Claude Code. It registers handlers for events (a tool call, a prompt, a part of the interface about to be drawn), and each handler can watch the event, rewrite it, or answer it so the usual behaviour never runs. Mods were introduced in Claude Code 2.1.287. They are JavaScript or TypeScript, and they need three files: a manifest, a *hooks.json* that points at your module, and the module itself.

The older settings hooks can block a tool call too, but they run as a shell script outside Claude Code and cannot draw anything. A mod can hold the call, run a program, keep state across the session, and put the result on screen. For this project the drawing part is the whole point.

The mod is one `tool.call` hook on Bash and one `ui.render` hook on the band above the prompt, plus a `/budget` command. The Bash hook checks the commit before it calls `next`. If the check fails, the commit does not run.

```
on('tool.call', { tool: 'Bash' }, async ($, e, next) => {
  if (COMMIT.test(e.command)) {
    const last = (await read($, samples)).at(-1)
    const over = last ? overBudget(last, o) : null
    if (over) return { deny: `firmware-budget refused this commit: ${over}` }
  }
  const ran = await next(e)
  if (isBuild(e.command, build) && !ran.deny && !ran.isError) {
    await measure($, o, e.command)
  }
  return ran
})
```

Before you install a mod, `claude plugin validate` reads its source the way the engine will and lists every event it hooks and every call it makes. For this mod the list is short - it runs one program, the `size` tool. You can (and should) check that in the public source code.

ELF Complications

The first version of the parser read Berkeley `size` output, the usual `text data bss` values, and took flash as `text + data` and RAM as `data + bss`. On my test ELF it reported data 0. Which was wrong: the firmware had 6,968 bytes of initialised data.

The cause is the Pico SDK linker script, which marks `.data` executable. Berkeley size classifies by section flags, so it folds `.data` into the `text` column and the `data` column reads zero. Flash comes out right and RAM is under-reported by the size of `.data`.

The mod now runs `size -A`, which lists every section with its address, and splits by where the linker put things. Anything below `0x20000000` is the XIP image in flash. Anything at or above it is RAM, which on the Pico SDK includes the vector table, `.data`, `.bss` and the SDK's minimum heap and stack reservations. The `.data` load image is added to flash, since it is stored there before boot copies it across. Debug sections have address zero and count for neither. The RAM start address is a setting, so an STM32 or nRF52 project is a configuration change rather than a fork of the code.

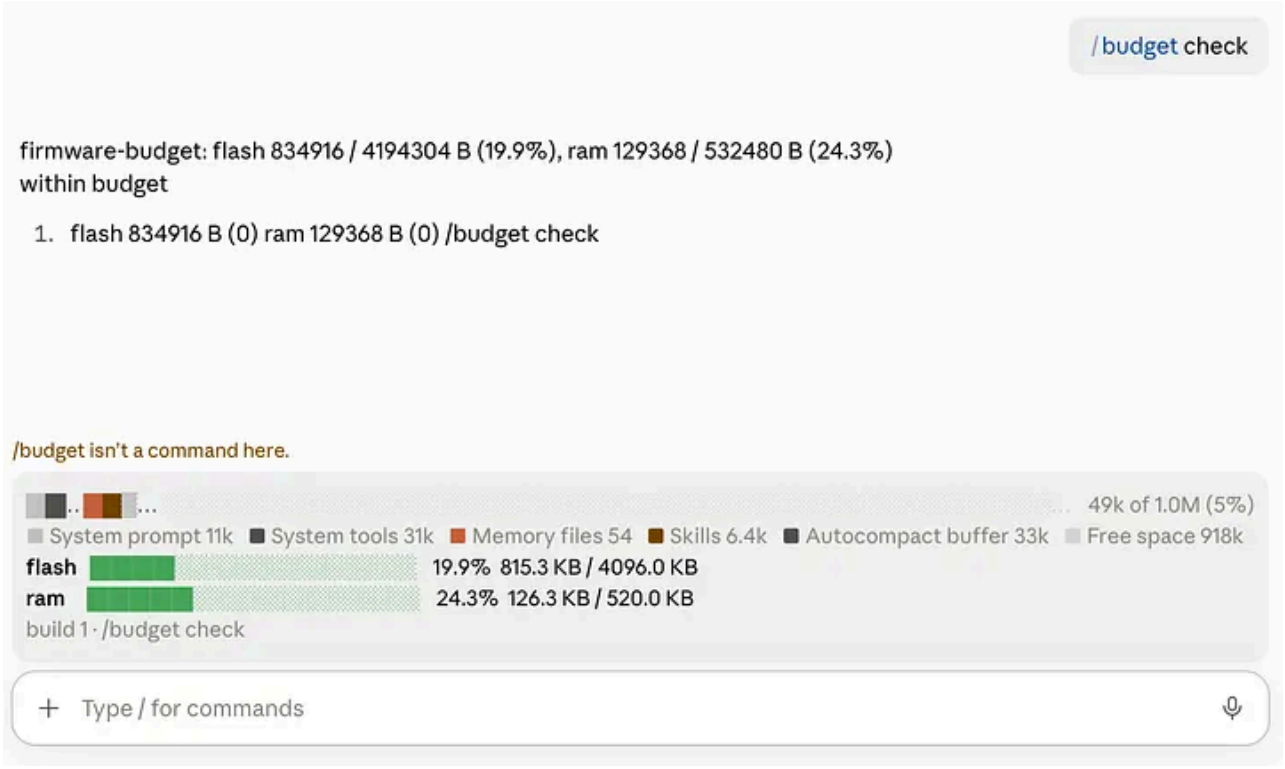
Three More Things That Went Wrong

The mod passed validation and its eight tests, but wouldn't draw the bars. There were three problems. Each failure was silent, and took a fair amount of effort to track down. So to save you having to make the same journey, here is what to look out for if things don't work.

- **The version.** My Claude Code was 2.1.233. Mods need at least 2.1.287. An older build reads the folder as a plugin with a manifest and no components, loads it, and does nothing. No error, no warning. `claude --version` is the first thing to check, and `claude plugin test` not being a recognised command is the second warning.
- **The PATH.** A mod runs with the session's environment. In the Desktop app that is the app's environment, not your shell's, so nothing from `~/.zshrc` is there. The Pico SDK installer also keeps its toolchain under `~/.pico-sdk` and off the `PATH` entirely. The first test failed with an `Executable not found in $PATH: arm-none-eabi-size`. The fix is putting the full path in the mod's settings. The lesson is that any mod that shells out to a toolchain needs the path as a setting.

- **The display band.** With the `size` tool found, `/budget check` printed the right numbers and nothing appeared above the prompt. I already run a context-usage bar in that band, from an earlier article. The display band is one site shared by every mod, and the mods on it form a chain: each hook draws its own tree and calls `next(e)` to let the ones behind it draw. If a hook returns without calling `next`, the chain ends there. My context bar was doing exactly that.

In order to fix this I needed to look at the debug log. Every band render had an entry reading context-bar answered `ui.render` without calling `next()`; nothing beneath it ran for this dispatch. Once the context bar called `next`, both mods worked correctly:



Claude Code Screenshot showing the mod (macOS desktop app)

Open in app ↗

Medium Search Play Games Write

Try It

The mod is MIT-licensed and available at Reefwing-Software/firmware-budget. Install it from inside Claude Code 2.1.287 or later using:

```
/plugin install firmware-budget --marketplace Reefwing-Software/firmware-budget
```

The mod asks for the ELF path at install. The memory budgets default to a Pico 2 (4 MB flash, 520 KB SRAM) and the RAM start address of `0x20000000`. An RP2040, STM32 or nRF52 project will require this changed using `/config`. Set the `size` tool to a full path if your toolchain is not included in the `PATH`, which on the Pico SDK it will be.

My first book, Embedded AI, is available now from No Starch Press. It includes 33 hands-on hardware projects deploying machine learning on microcontrollers. Sign up to my newsletter for updates, free article links, and bonus material! To support my writing here, please show your appreciation by following me, or subscribe to get an email whenever I publish a new article.

Embedded AI: Bonus Projects and Free Technical Articles

Embedded AI: Intelligence at the Deep Edge is now available from No Starch Press. Subscribe to the Embedded AI...

embedded-ai.kit.com

Embedded Ai + Claude + Mods + Ai Development +

Claude Code +



Written by David Such

5.4K followers · 2.4K following

Follow

Reefwing Software · Embedded Systems Engineer · iOS & AI Development · Robotics · Drones · Arduino · Raspberry Pi · Flight Control



Published in AI Advances Briefs

200 followers · Last published just now

Follow

Concise ideas. Serious thinking about AI. The short-form publication from AI Advances.

No responses yet



Alex Mylnikov

What are your thoughts?